Simple Dictionary — Beginner Study Guide

Read this short guide first. Then open CodeWalkthrough.html in a browser for the detailed file-by-file, line-by-line explanation. The walkthrough includes the source beside its explanation and a search box. No Internet connection is needed to read it.

The line references describe the code snapshot used to create this guide on 28 October 2026. Editing code can change its line numbers. This guide explains the implementation; it does not establish that every feature has passed device testing.

1. The app in one sentence

You enter a word, the app checks its saved data or asks online dictionaries, and the screen displays the definitions. Successful searches and favorites are saved so they can be reopened later. A separate setting remembers whether slang should start expanded.

2. Who does what?

Think of each layer as a person with one job.

Part	Simple job	Main files
UI	Shows information and reports button taps.	InputScreen.kt, ResultScreen.kt, HistoryScreen.kt
App	Connects the objects and decides which screen opens.	App.kt
ViewModel	Decides the current screen state and starts operations.	DictionaryViewModel.kt, UiState.kt
Domain	Describes the data and operations without knowing the screen or database implementation.	Models.kt, repository interfaces and use cases
Repository implementation	Decides whether to read saved definitions or request online definitions.	data/DictionaryRepository.kt
Room database	Keeps saved searches, definitions and favorites.	HistoryEntity.kt, HistoryDao.kt, DictionaryDatabase.kt
DataStore	Keeps the small Show slang by default setting.	DictionaryPreferences.kt
Platform setup	Starts the app and supplies the correct	Platform entry points, HttpClientFactory

Part	Simple job	Main files
	network engine and storage location.	files, AppStorage files

There are two DictionaryRepository.kt files. domain/DictionaryRepository.kt is the promise: which operations exist. data/DictionaryRepository.kt contains the actual work that fulfills that promise.

3. Learn these Kotlin words first

Code	Plain meaning
val / var	val cannot be reassigned; var can be reassigned. An object held by val can still contain changeable values.
class / data class	A class describes an object. A data class is a convenient container for related values.
interface	A list of operations an implementation promises to provide.
private	Only this class/file can access the declared detail, depending on where it is declared.
fun / return	Define an operation / send its answer back to the caller.
suspend	This function can pause without blocking the thread. It is called from a coroutine or another suspend function. It does not automatically move work to a background thread.
scope.launch	Start a coroutine. Other work can continue while it waits.
Flow	A stream that can send values over time.
StateFlow	A stream with a current value, suitable for screen state.
SharedFlow	A stream used here for messages that happen once, such as a snackbar notification. With this configuration, messages are not replayed to later collectors.
sealed class	A limited family of types, such as Loading, Success and Error.
when / if	Choose the matching case / check a condition.
try / catch	Attempt work and handle an error if it fails.

Code	Plain meaning
null / String?	No value / a String that is allowed to have no value.
?. / ?:	Continue only when the value exists / otherwise use the fallback value.
it	The single unnamed input of a short callback. Its meaning depends on the callback.
{ ... }	A block of code. When passed to another function, it is often a callback: work to perform later or during configuration.
@Composable	A function that describes part of the Compose screen. It can run again when observed state changes.
remember	Keep an object/value across redraws at this composition location. It does not save it to disk.
collectAsState()	Turn a Flow/StateFlow into a value Compose observes so the screen updates.
@Serializable	Allow Kotlin serialization to convert this data type to/from JSON.
expect / actual	Shared code declares what is needed; each platform supplies its implementation.
Modifier / dp / sp	Layout instructions / a layout size unit / a text-size unit.

4. Follow a new search: hello

The entry paths below are relative to shared/src/commonMain/kotlin/com/example/dictionary unless stated otherwise.

1. InputScreen.kt line 51 displays the text field. Lines 52–53 connect its displayed value and typing callback. The query is owned by App, not secretly stored in the text field.
2. App.kt line 62 keeps searchQuery in Compose memory. Line 136 changes that value when typing occurs. Typing alone does not call an API.
3. InputScreen.kt line 74 checks that input is not blank, then calls onSearchClick(query). It reports the action to App.
4. App.kt line 137 navigates to the selected word. Lines 74–85 build a route such as result/hello and ask the navigation controller to open it.
5. App.kt lines 146–151 define the result/{word} destination and read hello from the route. Lines 152–155 start the lookup when this destination is entered for that word.
6. DictionaryViewModel.kt lines 59–66 cancel a previous search, start a coroutine, set Loading, and call SearchWordUseCase. Cancelling prevents an earlier search from replacing a later result.

7. domain/SearchWordUseCase.kt lines 3–5 remove outside spaces and lowercase the word, then call the repository interface. For example, ' Hello ' becomes 'hello'.
8. data/DictionaryRepository.kt lines 40–45 ask Room whether hello is saved. For this example, assume it is not.
9. The repository requests Urban Dictionary at lines 48–55 and Merriam-Webster at lines 74–90. These requests are sequential: Urban first, formal dictionary second. Each has its own error handling.
10. The repository converts online responses into DictionaryEntry objects. The APIs can send different field names; the app converts them into its own common model.
11. Repository lines 115–120 package the results and save useful definitions using dao.saveWord(). JSON is used to fit the definition lists into one database text column.
12. The result returns through the use case to the ViewModel. ViewModel lines 104–109 select Success or a no-definitions Error.
13. App.kt line 64 observes uiState. ResultScreen.kt line 64 chooses what to display using when(state). Loading shows a spinner; Success shows definitions; Error shows a message.
14. Separately, Room's history Flow emits its updated list. The repository maps it at lines 126–133; the ViewModel collects it at line 41; App observes it at line 65; HistoryScreen displays it at lines 50–55.

The callback starts the trip toward the data. The returned result and observable state bring the answer back to the screen.

```
Typing -> App.searchQuery -> InputScreen redraws
Search tap -> App navigation -> word route
word route -> ViewModel -> use case -> repository
repository -> Room cache OR Ktor APIs -> saved result
result -> ViewModel.uiState -> App.collectAsState -> ResultScreen
Room history Flow -> ViewModel.searchHistory -> HistoryScreen
```

5. What changes for a saved word?

Repository lines 40–45 return the saved JSON as a DictionaryResult with isCached=true. The ViewModel displays it immediately at line 75. Lines 77–89 then request fresh definitions without clearing the already displayed cached result. If refreshing fails, a message is sent while the cached definitions remain visible.

Offline does not mean the entire dictionary is installed. It means previously saved words remain available. A new uncached word still needs the Internet.

The Refresh definitions button is in ResultScreen.kt lines 56–59. App.kt line 161 calls searchWord with forceRefresh=true, which bypasses the repository's initial cache check.

6. What happens when you tap a star?

1. HistoryScreen.kt line 86 calls onToggleFavorite(item.word).
2. App.kt line 169 forwards the word to the ViewModel.
3. DictionaryViewModel.kt lines 112–120 call HistoryUseCase inside a coroutine and report a storage error if necessary.
4. HistoryUseCase.kt lines 8–9 call the repository interface.
5. data/DictionaryRepository.kt lines 136–137 call the DAO.
6. HistoryDao.kt lines 27–28 flip the saved favorite flag.
7. The DAO's history query, lines 9–10, sends the changed list with favorites sorted first. The Flow path redraws the history automatically.

The star is stored in Room. It is not just a temporary color change in the UI.

7. What happens when you change the slang setting?

1. InputScreen.kt line 65 reports the checkbox's new Boolean value.
2. App.kt line 139 forwards it to saveShowSlang().
3. DictionaryViewModel.kt lines 124–132 call SlangPreferenceUseCase and send a saved/failure message.
4. SlangPreferenceUseCase.kt lines 8–9 call the preferences interface.
5. DictionaryPreferences.kt lines 19–20 use DataStore.edit to save show_slang.
6. Its data.map at lines 15–16 observes the saved value; the ViewModel collects it at line 50; App observes it at line 66.
7. ResultScreen.kt line 28 uses the saved default when initializing its local show/hide state for a word. Manually expanding/collapsing the current result does not itself save a new default.

Room keeps the dictionary data. DataStore keeps the small setting. remember keeps temporary UI values only for the current composition lifetime.

8. Reading the database files

HistoryEntity.kt is the shape of one row. word is its primary key, so the same normalized word is represented by one row. snippet is the history preview; resultJson contains serialized definitions; isFavorite is the star; searchOrder helps sort recent saved results.

HistoryDao.kt contains the database operations. SELECT reads rows. ORDER BY puts favorite rows first and higher searchOrder next. INSERT adds a word. ON CONFLICT(word) updates the saved definitions if that word exists. Notice that this update does not replace isFavorite. UPDATE ... NOT isFavorite flips the star.

DictionaryDatabase.kt registers the row type and exposes the DAO. The expect constructor is fulfilled by Room-generated code. You do not hand-write the generated constructor files.

AppStorage.kt groups the database and preferences objects. The platform-specific AppStorage.kt files choose a real location and build those objects. Desktop uses ~/.kmp-dictionary; Android uses private app directories; iOS uses Documents; browsers use browser storage.

9. Which module should we mention in the defense?

Module	Show these files	Explain in your own words
6	Repository interface, use cases, App.kt lines 47–58, source-set dependencies and platform factories	We separate jobs. Objects receive their dependencies. Shared code asks for a platform client through expect/actual.
7	UiState.kt and DictionaryViewModel.kt lines 25–50	One state describes the result screen. StateFlow holds current values. SharedFlow carries one-time messages. Coroutines let requests wait without freezing normal UI work.

Module	Show these files	Explain in your own words
8	App.kt lines 64–66 and 128–169; ResultScreen.kt line 64	Compose observes state. NavHost opens screens and carries the selected word in the route.
9	HttpClientFactory.kt, data/DictionaryRepository.kt, HistoryDao.kt and DictionaryPreferences.kt	Ktor gets typed online data; errors are handled. Room saves definitions/favorites. DataStore saves the setting. Cached data works offline.

10. Extra setup: read this after the main app

The Android Activity, desktop main(), iOS MainViewController and browser main() are platform doors into the shared App. They create storage and show the shared UI.

Gradle files are build instructions, not functions that run every time you search. The version catalog records dependency versions. AndroidManifest.xml declares Internet permission and the Android deep-link URL pattern. Browser index.html/styles.css supply the webpage around Compose.

browserStorage contains the browser SQLite bridge. Kotlin sends database requests to a JavaScript worker. The worker opens a browser file, prepares SQL, runs it and replies with rows. This infrastructure is additional to the modules' short examples; it is isolated so the shared app logic stays easier to study. Its original source and license are retained.

PersistenceTest.kt checks database reopening, favorites, refresh behavior, invalid API responses and DataStore reopening with mocked HTTP responses. NavigationCompatibilityTest.kt creates a small rendered navigation host and checks opening a word and going back. These files check behavior; they are not code that handles a normal user's search.

The app previously crashed because an old Navigation library expected a saved-state method that newer dependencies did not have. Updating Navigation and using the compatible argument-reading API fixed that mismatch. A dependency change needs a full app restart; a hot reload does not replace the process's dependency libraries.

11. Things to understand rather than memorize

A use case requests work; the repository implements data access. A suspend function can pause; it does not promise a new thread. A Flow is observed repeatedly; a normal function return gives one answer. A UI callback does not save data until the receiving code calls a storage operation. JSON from an API, a Kotlin model and a database row are related but are not the same representation.

The source is an implementation of the module concepts, not a copy of every PDF example. Room bootstrap, platform paths, timeout/cancellation handling, the navigation compatibility API and the browser worker need additional library/platform details.

12. Suggested study order

1. Read sections 1–3 together.
2. Trace hello using section 4 and the walkthrough's line links.
3. Read Models.kt, UiState.kt and SearchWordUseCase.kt.

4. Read DictionaryViewModel.kt, then data/DictionaryRepository.kt.
5. Trace the star and setting without looking at a prepared answer.
6. Read the database and platform files.
7. Each member should practice explaining a line chosen by another member and describe what would happen if its input changed.

Open CodeWalkthrough.html for the complete source-line reference. Imports, empty lines and closing brackets are explained too, but can be hidden while studying the actual behavior. Assets such as launcher PNGs and generated/build files are listed as supporting files rather than treated as handwritten Kotlin logic.